

# CIS 3200 – Homework 2

Sunny Lee  
lseonu@seas.upenn.edu

July 17, 2026

Partners: Yeongheon Lee, Josh Lee.

## Problem 1

### Algorithm:

Let servers be represented as  $S_i$ ,  $1 \leq i \leq n$ .

- **Step 1:** Scan through the set  $S$  and find the minimum( $B$ ) and maximum number( $A$ ), by initializing  $A$  as the first element, then when checking the next element, compare it with  $A$  and if the next element is greater than  $A$ , update  $A$  to the next element, then compare with  $B$ , if the next element is smaller than  $B$ , update  $B$  to the next element, and iterate through the whole set to find  $A$  and  $B$ .
- **Step 2:** Initialize an element,  $c$ , to equal  $\frac{A-B}{n-1}$ .
- **Step 3:** Use radix sort, as learned in CIS 3200 01/27 lecture, so that the sorted array,  $A$ , has  $A[1] = B$  and  $A[n] = A$ .
- **Step 4:** For all  $n$  elements in the sorted array  $A$  ( $1 \leq i \leq n-1$ ), for each adjacent pairs, check if  $A[i+1] - A[i]$  is less than or equal to  $c$ . If so, return the adjacent pair  $A[i+1]$  and  $A[i]$ .

### Proof of Correctness:

Observation: Note that there are  $n$  distinct elements within the interval of  $[B, A]$ . That is, there are exactly  $n-1$  gaps between the elements when we sort it. Because there are  $n$  numbers and  $n-1$  gaps, we know by the pigeonhole principle that there must exist a pair of similar integers, with a gap less than or equal to  $\frac{A-B}{n-1}$ .

Now, we know that step 1 would correctly identify the minimum and maximum element in the set as we iterate through the array and ensure that the maximum and minimum is updated after comparison each iteration. We also know that using radix sort, we can ensure that we sort the array in ascending order, as we learned in lecture. Once we have a sorted array from step 3, we can scan through adjacent pairs and ensure that we find a pair of similar integers since the array is sorted, thus the pairs with smallest difference in value must be adjacent pairs.

### Running Time:

Steps 1 takes  $O(n)$  time since we are scanning through  $n$  elements and comparing and updating the variables  $A$  and  $B$ , which takes constant time. Step 2 takes constant time since it is a simple arithmetic operation. Step 3 takes  $O(n)$  time, as that is the runtime of radix sort as learned in lecture. Step 4 takes  $O(n)$  time as well, as we are iterating through the sorted array of  $n$  elements

once, checking the adjacent pairs and comparing the difference between the two integers of the adjacent pair with  $c$  (which takes constant time).

Therefore, the total runtime is  $O(n)+O(1)+O(n)+O(n)= O(n)$ .

**Problem 2****Algorithm:**

- **Step 1:** Create two hash tables  $T_1$  and  $T_2$ , that uses a universal hash function, to save frequency of each character in string A and B, respectively. The size of the two hash tables is  $200n$ .
- **Step 2:** For each character in A and B,
  - Universal hashing and the universal hash function would be used to hash the characters in A to the table  $T_1$  and that of B to the table  $T_2$ .
- **Step 3:** Check if the two tables are the same, if yes, return YES and if not return NO.

**Proof of Correctness:**

We want this algorithm to correctly output the result with a probability of failure of less than 1 percent. We know from lecture that the probability of collision of keys is  $\frac{1}{\frac{2^n}{\text{cellcount}}}$ . Now, since we have at most  $2n$  keys, and the probability of collision is maximum  $\frac{2n}{\text{cellcount}}$ . Therefore, to ensure that the probability of failure is less than 1 percent, the probability of collision must be less than  $\frac{1}{100}$ , thus the cell count must be greater than  $200n$ . Also, note that the two strings would be anagrams if the two tables are the same, as the hash function used is the same, thus the result of mapping must be the same.

**Running Time:**

Creating the hash table and hashing through  $n$  characters each in A or B (using a universal hash function), takes  $O(n)$  time. Comparing each cells in the two tables take constant time per cell, and there are  $O(n)$  cells, so the total runtime is  $O(n)$  time.

**Space Complexity:**

We have two tables of size  $200n$ , so the space complexity is  $O(n)$ .

**Problem 3****Algorithm:**

- **Step 1:** Create a table,  $M[i][x][y]$  that denotes the maximum total tip from the  $i$  number of orders, when  $x$  is the number of orders delivered by Alice and  $y$  is the number of orders delivered by Bob. Initialize  $M[0][0][0]$  to 0 and for all other  $x$  and  $y$  values when  $i=0$ , set  $M[0][x][y] = -\infty$ .
- **Step 2:** For each order  $i$ , set  $M[i][x][y] = \max(M[i-1][x][y], M[i-1][x-1][y] + A[i], M[i-1][x][y-1] + B[i])$ .
- **Step 3:** After iterating through all orders find the maximum value over all and output the maximum value.

**Proof of Correctness:**

We will prove this using induction.

Base Case: We know that when  $i = 0$ , there is no orders, therefore, nobody would have delivered orders, and therefore, the tip must be 0. Thus, this algorithm correctly outputs 0.

Induction Step: Our induction hypothesis is that the algorithm correctly outputs the maximum tip for all  $x$  and  $y$  for the first  $i-1$  orders, thus  $M[i-1][x][y]$  correctly denotes the maximum tip. Now, for order  $i$ , we know that there are three cases possible: 1. Skip the order, 2. Alice delivers it, 3. Bob delivers it. In the first case, the maximum tip would be exactly the same as  $M[i-1][x][y]$ . In the second case, the maximum tip would be  $M[i-1][x-1][y] + A[i]$ . In the third case, if Bob delivers it, the maximum tip is  $M[i-1][x][y-1] + B[i]$ . Therefore, by taking the maximum, we know that we correctly find the maximum value in that case. Finally, since we calculate all cases and find the maximum value over all in step 3, we can ensure that we could always ensure that we find the maximum total tip given all constraints.

**Running Time:** We are creating a table of  $(n+1) \times (a+1) \times (b+1)$ . Now, iterating through each state and updating the table would take constant time each, and there are  $(n+1) \times (a+1) \times (b+1)$  states so the total time would be  $O(nab)$  time.

**Problem 4****Algorithm:**

- **Step 1:** Create a table,  $M[i][j]$ , that denotes the side length of the largest square, starting from the bottom right corner and set  $M[1][1] = A[1][1]$ .
- **Step 2:** For  $A[i][j] = 0$ , set  $M[i][j] = 0$ , and for all other cells ( $i \geq 1$  and  $j \geq 1$ ), if  $A[i][j] = 1$  set  $M[i][j] = \min(M[i-1][j], M[i][j-1], M[i-1][j-1]) + 1$ .
- **Step 3:** After iterating through all cells, find the maximum value over all and save this value as maximum.
- **Step 4:** Return the square of maximum value found in step 3.

**Proof of Correctness:**

We want to prove that this algorithm finds the maximum side length of the square that ends at each cell. Let's prove this using induction.

Base Case: We know that  $M[i][j] = A[i][j]$  when  $i=1$  or  $j=1$ , because if  $A[i][j] = 1$ , then the maximum side length is 1 and if  $A[i][j] = 0$ , the maximum side length would be 0 since there are no squares of 1. Thus, the claim holds true.

Induction Step: Our induction hypothesis is that for all  $m \times n$  cells ( $m \geq i, n \geq j$ ),  $M[m][n]$  denotes the maximum side length of square ending at  $(m, n)$ . We know that for cell  $(i, j)$ , there are two possible cases: 1. if  $A[i][j] = 1$ , then the  $(i,j)$ th cell might be the bottom right corner of the square. Then, the squares that end at  $(i-1, j)$ ,  $(i, j-1)$ ,  $(i-1, j-1)$  must also be in the square. Thus, by taking the minimum and adding 1, we could correctly find the maximum side length ending at  $(i,j)$ . The second case possible is that  $A[i][j] = 0$ , which means that there are no square ending at  $(i,j)$ , thus  $M[i][j] = 0$ , which is correct. Thus, we know that we could always ensure that we find the length of maximum side length of square, and in step 4, since we return the square of maximum value, we can correctly find the area of maximum square.

**Running Time:** There are  $m \times n$  cells in the matrix, and for each cell, the computation takes constant time (step 2). Therefore, the total runtime is  $O(mn)$ .

**Problem 5****Algorithm:**

- **Step 1:** Initialize an array  $A$ , with size  $n$ , and set all elements to equal 1.
- **Step 2:** For each employee  $i$  ( $2 \leq i \leq n$ ), if the performance of the current employee( $i$ ) is better than that of the adjacent employee( $i-1$ ), set  $A[i] = A[i-1] + 1$ .
- **Step 3:** For each employee  $i$  ( $1 \leq i \leq n - 1$ ), starting from  $n-1$ , if the performance of the current employee( $i$ ) is better than that of the adjacent employee( $i+1$ ), set  $A[i] = \max(A[i], A[i+1] + 1)$ .
- **Step 4:** Return output array  $A$ .

**Proof of Correctness:**

Step 1 ensures that every employee gets at least 1 bonus point, as stated in problem. Step 2 ensures that condition 2 (to minimize envy, it is required that if an employee has a strictly higher performance rating than an immediate neighbor, then they must get assigned strictly more bonus points than that immediate neighbor) is true when comparing with the left adjacent employee. In step 3, we compare with the right adjacent employee and ensure that condition 2 is kept, while also making sure that we always pick the value with enough bonus points for both the left and right check by using  $\max$ . This ensures that we start with the minimal bonus, then increase at only necessary points, thus minimizing the total bonus while keeping the conditions.

**Running Time:**

Step 1 and 4 takes constant time, and the comparison in steps 2 and 3 takes constant time, but we are repeat this comparison for  $n$  elements in step 2 and again  $n$  elements in step 3, thus step 2 and 3 both take  $O(n)$  time. Therefore, the total run time is  $O(n)$ .